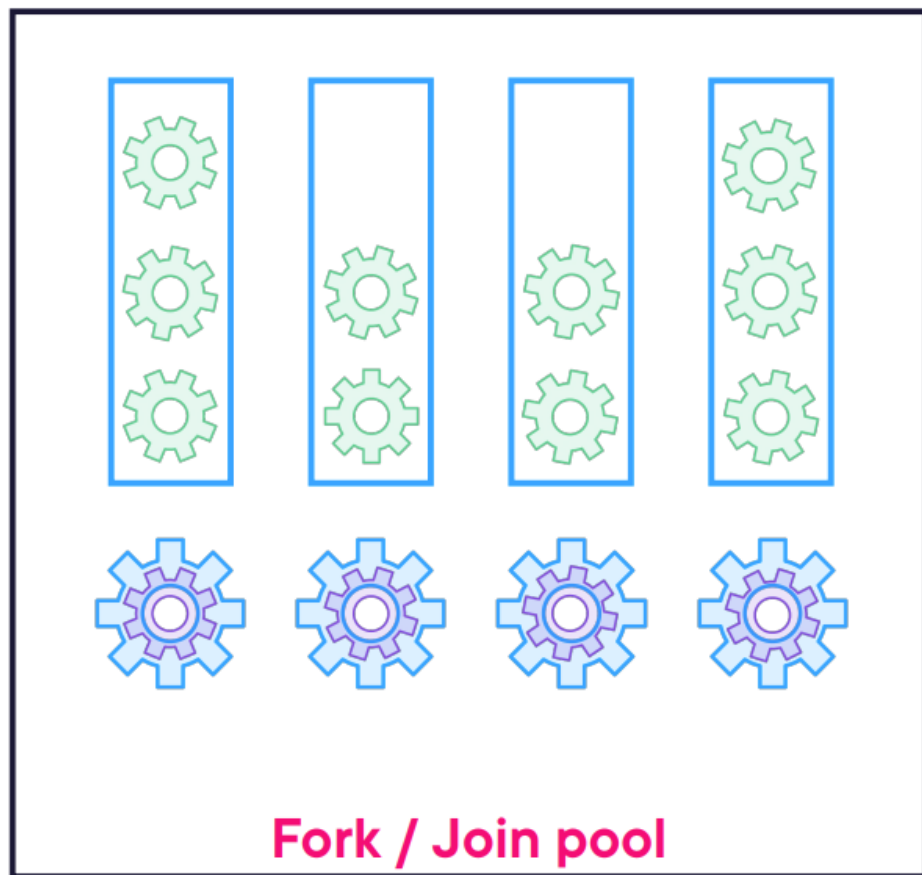


深入理解虚拟线程

北京理工大学计算机学院
金旭亮



虚拟线程与Fork/Join pool



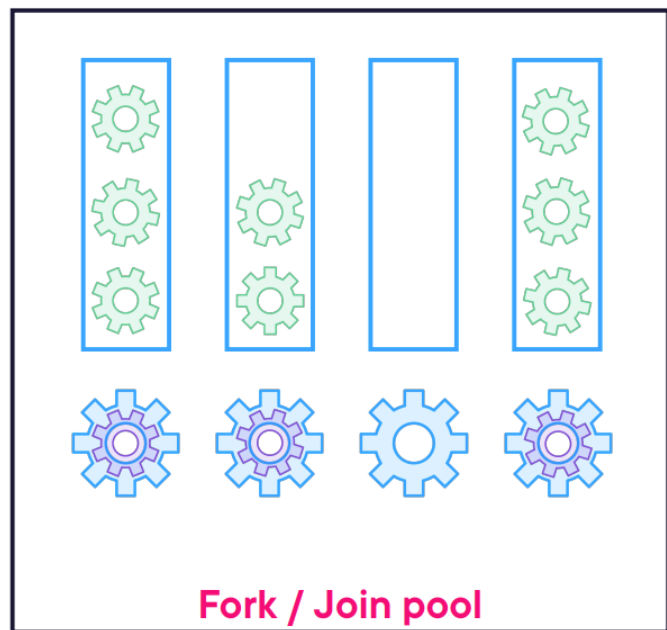
虚拟线程并不是一个“真正”的线程，它自己是不能执行的，需要“承载于”一个真实的“平台线程”才能运行，这个“平台线程”，不妨称之为“工作线程（worker thread）”，来自于一个特殊的线程池——Fork/Join Pool，如左图所示。

池中的每个工作线程，都关联有一个任务队列，队列中的“任务”，其实就是虚拟线程需要执行的“任务”，是以虚拟线程函数代码的方式表达的。

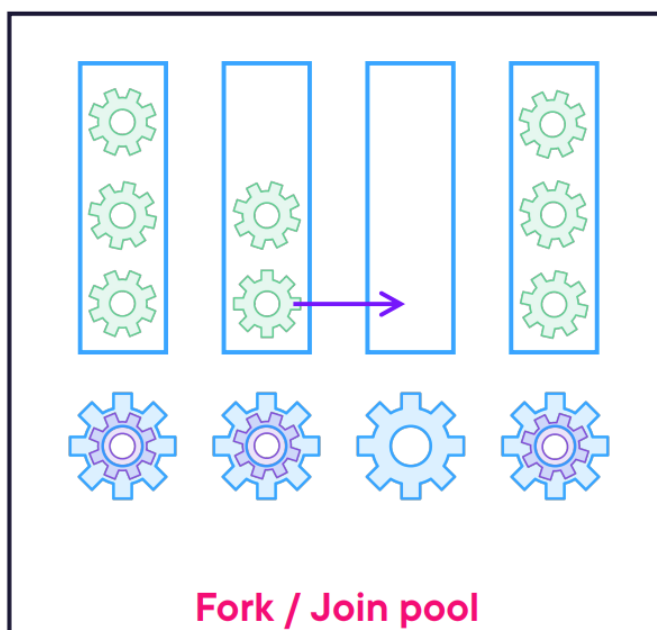
任务窃取



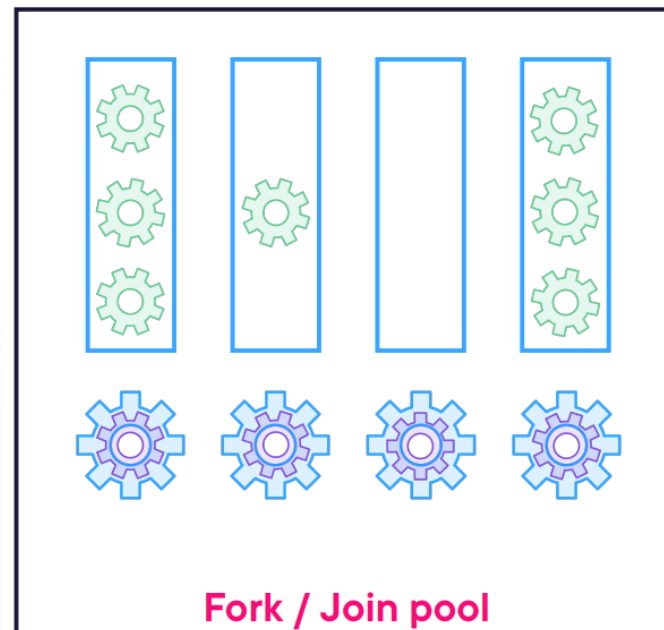
Fork/Join Pool内置负载均衡机制，当某个平台线程工作繁忙时，其他“空闲”的线程会主动过来“帮忙”，从其工作队列中“偷”一个任务过去处理。



第3个线程“空闲”



第3个线程从第2线程任务队列中“偷”了一个任务



第3个线程执行任务，第2个线程少了一个任务负担

直观了解虚拟线程与工作线程-1



MillionVirtualThreads.java

```
//保存线程池的名字
Set<String> poolNames = ConcurrentHashMap.newKeySet();
//保存线程池中工作线程的名字
Set<String> pThreadNames = ConcurrentHashMap.newKeySet();

//将创建100万个虚拟线程
int N_THREADS = 1_000_000;

var threads = IntStream.range(0, N_THREADS)
    .mapToObj(i -> Thread.ofVirtual()
        .unstarted(
            () -> {
                //获取线程池中虚拟线程的名字
                String poolName = readPoolName();
                poolNames.add(poolName);
                //获取平台线程的名字
                String workerName = readWorkerName();
                pThreadNames.add(workerName);
            }
        )
    ) .toList();
```

示例代码使用Stream API创建了100万个虚拟线程，将其名字加入到相应集合中，然后统计其执行时间。

100万个线程，如果是平台线程，会耗用大量的系统资源，但由于是虚拟线程，则不会有问题。

```
//启动所有虚拟线程，等待其运行结束，并计算运行时间
Instant begin = Instant.now();
for (var thread : threads) {
    thread.start();
}
for (var thread : threads) {
    thread.join();
}
Instant end = Instant.now();
```

直观了解虚拟线程与工作线程-2



```
System.out.println("# 虚拟线程个数 = " + N_THREADS);  
System.out.println("# 本机CPU核数 = " + Runtime.getRuntime().availableProcessors());  
System.out.println(" 耗费时间 = " + Duration.between(begin, end).toMillis() + "ms");  
System.out.println("### 线程池名字");  
poolNames.forEach(System.out::println);  
System.out.println("### 使用的工作线程个数: " + pThreadNames.size());
```

```
Run  MillionVirtualThreads x  
"C:\Program Files\Java\jdk-21\bin\java.exe"  
# 虚拟线程个数 = 1000000  
# 本机CPU核数 = 16  
  耗费时间 = 560ms  
### 线程池名字  
ForkJoinPool-1  
### 使用的工作线程个数: 16  
  
Process finished with exit code 0
```

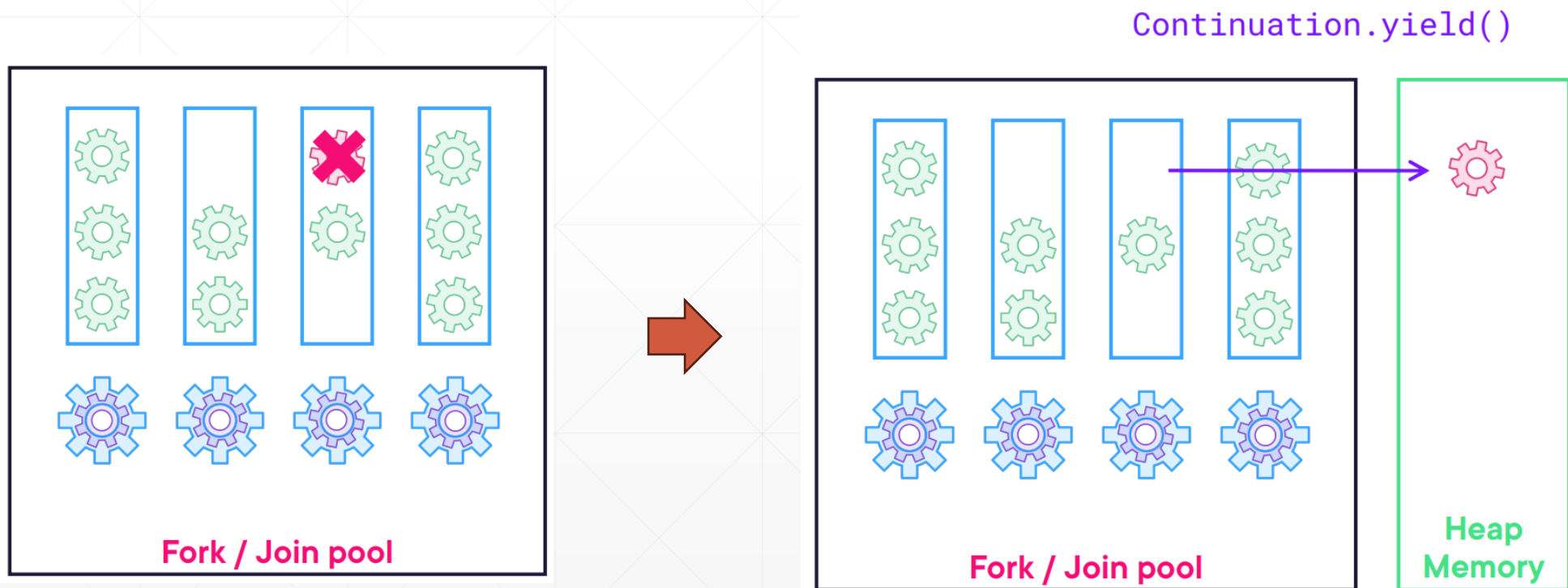
从输出中可以看到，应用程序可以创建很多的虚拟线程，但它所对应的工作线程（平台线程），只有有限个。

默认情况下，工作线程的个数，与本机CPU的核数是线性相关的。

虚拟线程的“挂起”与“恢复”-1



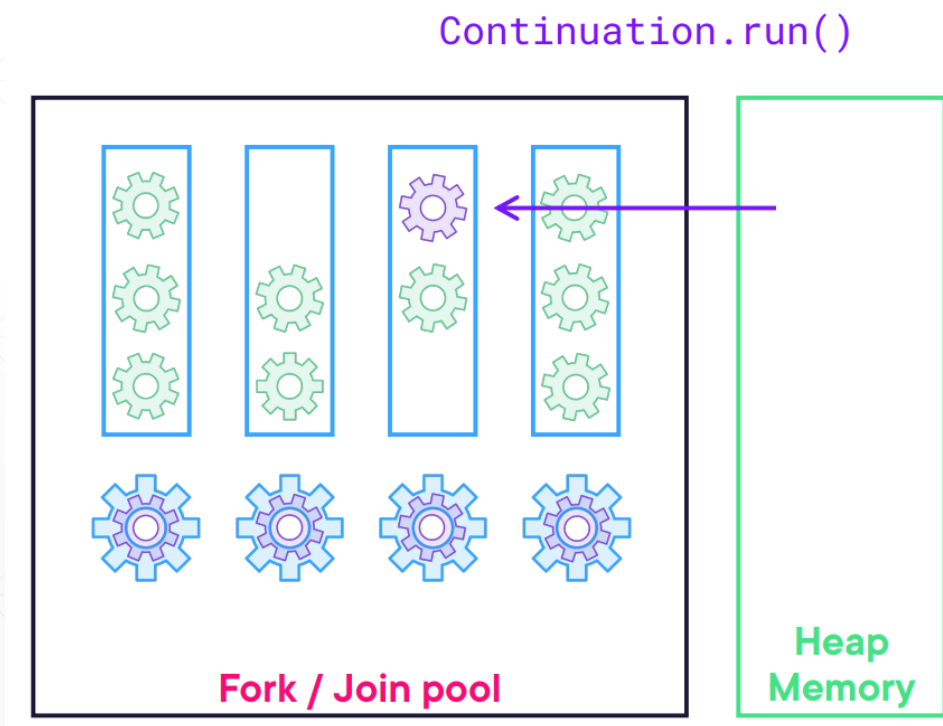
在底层，每个虚拟线程实际上是由一个Continuation对象负责执行的，当虚拟线程执行一个“阻塞操作（比如访问文件）”时，它会调用Continuation.yield()方法“释放”当前的工作线程，相关的执行上下文信息，被移到“堆”中。



虚拟线程的“挂起”与“恢复”-2



等到I/O操作完成，调用`Continuation.run()`方法，工作任务又会“回到”工作线程的工作队列中，继续等待执行.....



直观了解“挂起”与“恢复”



```
public class PlayWithContinuation {  
    public static void main(String[] args) {  
        //任务的“挂起”与“恢复”，是在一个ContinuationScope中完成的  
        var scope = new ContinuationScope("My scope");  
        var continuation = new Continuation(  
            scope,  
            () -> {  
                System.out.println("continuation开始执行，之后调用yield()方法‘挂起’");  
                //这里“挂起”，释放当前工作线程  
                Continuation.yield(scope);  
                System.out.println("‘挂起’结束，恢复执行");  
            });  
  
        System.out.println("main方法执行");  
        continuation.run();  
        System.out.println("回到main方法执行");  
        continuation.run();  
        System.out.println("回到main方法执行");  
    }  
}
```

```
Run  PlayWithContinuation x  
"C:\Program Files\Java\jdk-21\bin\java.exe"  
main方法执行  
continuation开始执行，之后调用yield()方法‘挂起’  
回到main方法执行  
‘挂起’结束，恢复执行  
回到main方法执行  
  
Process finished with exit code 0
```


工作线程的切换



在实际开发中，一些工作任务可能需要经历多次“挂起”与“恢复”的过程，比如，先访问互联网上的某个API，读取数据，进行处理之后，再发起另一个HTTP请求.....

如果使用虚拟线程来完成上述任务，默认情况下，Java是如何分派工作线程来完成上述任务的？

两个工作任务



```
Runnable multiPhaseTask =
    () -> {
        System.out.println("开始: " + Thread.currentThread());
        //第一阶段
        sleepFor(10, ChronoUnit.MICROS);
        System.out.println("第一阶段结束: " + Thread.currentThread());
        //第二阶段
        sleepFor(10, ChronoUnit.MICROS);
        System.out.println("第二阶段结束: " + Thread.currentThread());
        //第三阶段
        sleepFor(10, ChronoUnit.MICROS);
        System.out.println("所有任务全部完成:" + Thread.currentThread());
    };
```

```
Runnable otherTask =
    () -> {
        //不干啥正经事，空转，就为了模拟多任务的场景，
        int count=0;
        sleepFor(10, ChronoUnit.MICROS);
        do {
            count++;
        } while (count <= 100_000);
    };
```

```
//休眠指定长的时间
private static void sleepFor(int amount,
ChronoUnit unit) {
    try {
        Thread.sleep(Duration.of(amount, unit));
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

测试工作线程的切换情况



BlockingVirtualThreads.java

```
int N_THREADS = 10;
var threads = new ArrayList<Thread>();
//第一个虚拟线程执行multiPhaseTask, 其余的线程, 执行otherTask
for (int index = 0; index < N_THREADS; index++) {
    var thread = index == 0 ?
        Thread.ofVirtual().unstarted(multiPhaseTask) :
        Thread.ofVirtual().unstarted(otherTask);
    threads.add(thread);
}
//启动所有虚拟线程执行
for (Thread thread : threads) {
    thread.start();
}
//等待所有虚拟线程执行完毕
for (Thread thread : threads) {
    thread.join();
}
```

从输出中可以看到, 同一任务的不同阶段, 可以由不同的工作线程负责执行, 具体由哪个工作线程执行, 是由Java负责分派的, 开发者无需理会。

```
Run BlockingVirtualThreads x
开始: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-1
第一阶段结束: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-2
第二阶段结束: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-2
所有任务全部完成: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-11
Process finished with exit code 0
```

小结：Java虚拟线程的适用场景



如果要执行的任务是“CPU密集”的，并且不存在需要大量创建平台线程并行执行的需求，那么，虚拟线程由于需要在线程堆栈和进程“堆”中交换信息，还不如直接使用平台线程更高效。



但如果要执行的任务包容有I/O操作，那么虚拟线程的优势就很大了，因为虚拟线程不会阻塞工作线程，而会在等待IO操作时被临时放置到堆中（称为“挂起”），然后注册一个用于“恢复执行”的回调函数，这时，工作线程可以干其他的活，等到IO操作完成，虚拟线程会从上上次“挂起”的地方继续执行。



综上所述，可以知道，如果要执行的任务包容I/O操作，那么，使用虚拟线程是合适的。



Java虚拟线程最大的一个特点，就是即使它执行的任务本身是“阻塞的”（比如它需要访问网络），也不会阻塞执行它的“平台线程”。